Software Architecture and Design Specification

Project: Hospital Management System

Version: 1.0

Authors:

SRN	Name
PES1UG24AM435	Krishna Kaushik Reddy K
PES1UG24AM446	Rolla N Harsha Vardhan
PES1UG24AM458	Amulya P
PES1UG24AM921	Rohan Agarwal

Date: 09-09-2025

Status: Draft

Revision History

Version | Date | Author | Change Summary

1.0 | 30-09-2026 | —— | Initial SAD Document

2.0|14-10-2026| — —| Requirements Updation

Approvals

Role | Name | Signature/Date

1. Introduction

1.1 Purpose
This document specifies the architecture and design of the Hospital Patient Record and Appointment Management System. It describes the system architecture, major components, interactions, design decisions, security considerations, and interfaces of the proposed system.

1.2 Scope
The system provides a centralized platform for managing patient information, doctor information, medical records, and hospital appointments. Patients can manage their profiles and appointments, doctors can manage their availability and patient records, and administrators can manage users, doctors, appointments, and system records.

1.3 Audience
The system provides a centralized platform for managing patient information, doctor information, medical records, and hospital appointments. Patients can manage their profiles and appointments, doctors can manage their availability and patient records, and administrators can manage users, doctors, appointments, and system records.

- 1.4 Definitions
  Patient: A registered user who can manage personal information, appointments, and access permitted medical records.
- **Doctor:** A healthcare professional who manages availability, appointments, and relevant patient medical records.
- **Administrator:** A system user responsible for managing patients, doctors, appointments, and system-level information.
- **Appointment:** A scheduled consultation between a patient and a doctor.
- **Medical Record:** A patient's stored healthcare information and consultation history.
- **API:** Application Programming Interface used for communication between system components.
- **RBAC:** Role-Based Access Control used to restrict functionality based on user roles.

2. Document Overview

2.1 How to use this document
This document describes the architecture and detailed design of the Hospital Patient Record and Appointment Management System. It provides an overview of the system architecture, major components, interactions between components, security considerations, API design, and important design decisions.

The document can be used by developers to understand the system structure and implementation responsibilities, by testers to understand system interactions and requirements, and by project stakeholders to review the overall design of the system.

2.2 Related Documents
The following documents are related to this Software Architecture and Design Specification:

- Software Requirements Specification (SRS)
- Software Test Plan (STP)
- Requirements Traceability Matrix (RTM)
- UML diagrams

- Database/ER design

3. Architecture

3.1 Goals & Constraints

Goals

- Provide a centralized system for managing patient records and hospital appointments.
- Provide secure access to information based on user roles.
- Allow patients to efficiently book and manage appointments.
- Allow doctors to manage their availability and relevant patient records.
- Allow administrators to manage patients, doctors, appointments, and system information.
- Maintain consistency and reliability of stored patient and appointment data.
- Provide a modular architecture that is easy to maintain and extend.

Constraints

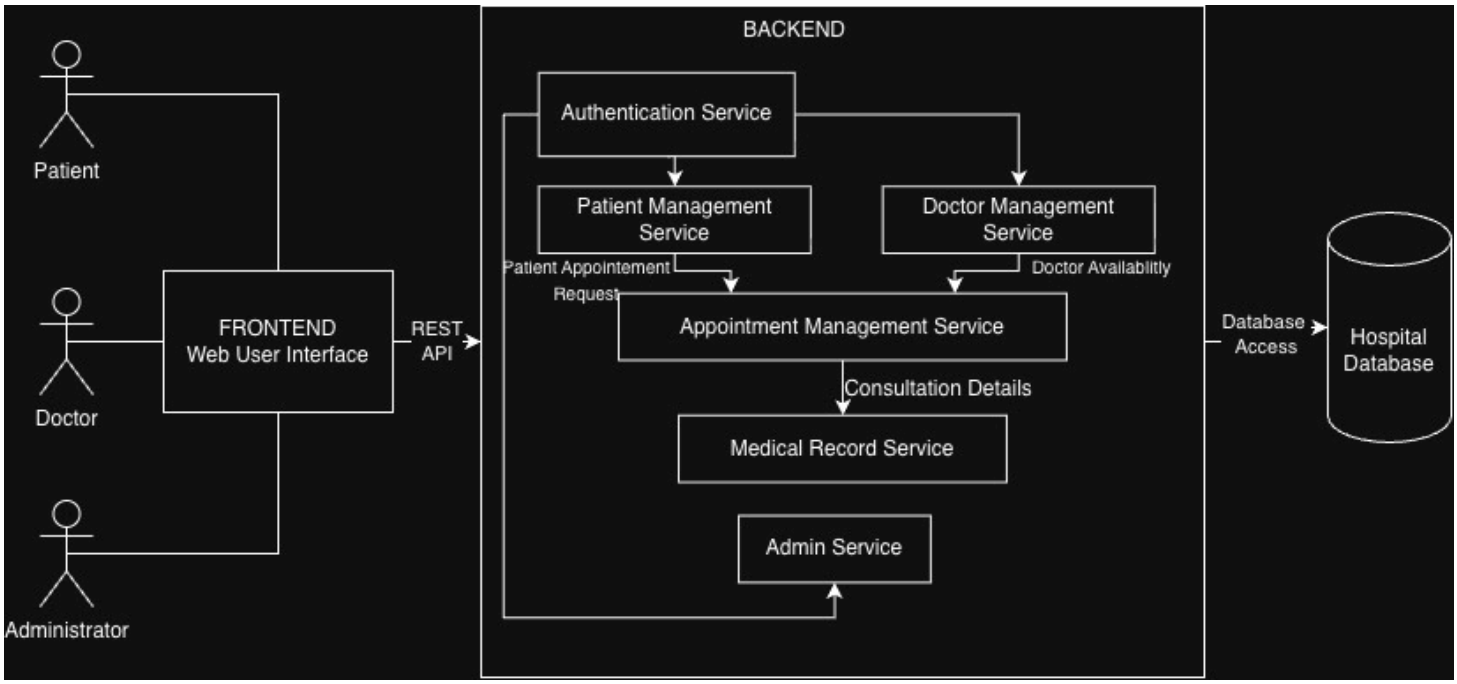
- Patient and medical information must be protected from unauthorized access.
- Access to system functionality must be controlled according to user roles.
- The system depends on a database for persistent storage of patient, doctor, appointment, and medical record information.
- The system requires network connectivity for communication between the frontend, backend, and database.
- The project must be feasible to develop and test within the given academic project timeline.

3.2 Stakeholders & Concerns

Stakeholder	Main Concerns
Patients	Easy appointment booking, accessibility, privacy of personal and medical information

Doctors	Easy access to appointments, availability management, and relevant patient records
Administrators	Efficient management of patients, doctors, appointments, and system data
Developers	Modularity, maintainability, scalability, and clear system interfaces
Testers	Testability, reliability, traceability, and consistent system behavior
Project Mentor	Correctness of requirements, architecture, design, implementation, and testing

3.3 Component (UML) Diagram



The system follows a layered component-based architecture. Users interact with the system through the frontend interface. The frontend communicates with backend services through defined APIs. The backend contains separate services for authentication, patient management, doctor management, appointment management, medical records, and administration. These services communicate with the database for persistent storage and retrieval of system information.

3.4 Component Descriptions

- **Frontend:** Provides the web-based user interface through which patients, doctors, and administrators interact with the system.

- **Authentication Service:** Handles user authentication and role-based access for patients, doctors, and administrators.
- **Patient Management Service:** Manages patient profiles and patient-related information.
- **Doctor Management Service:** Manages doctor profiles and doctor availability information.
- **Appointment Management Service:** Handles appointment booking, viewing, rescheduling, cancellation, and appointment status management.
- **Medical Record Service:** Manages the creation, updating, and retrieval of patient medical records and consultation information.
- **Admin Service:** Provides administrative functions for managing patients, doctors, appointments, and system-level information.
- **Hospital Database:** Provides persistent storage for user, patient, doctor, appointment, and medical record data.

3.5 Chosen Architecture Pattern and Rationale

The system uses a Layered Architecture pattern. The architecture separates the presentation, business logic, and data access responsibilities into distinct layers.

The presentation layer provides the user interface through which patients, doctors, and administrators interact with the system. The business logic layer contains services responsible for authentication, patient management, doctor management, appointment management, medical records, and administrative functions. The data layer manages persistent storage and retrieval of system information from the hospital database.

Layered architecture was chosen because it provides clear separation of concerns, improves maintainability, supports modular development, and makes the system easier to test and extend. It is also appropriate for the scope of the project without introducing unnecessary architectural complexity.

3.6 Technology Stack & Data Stores

Layer	Technology	Purpose
Frontend	React.js	Development of the web-based user interface
Backend	Node.js with Express.js	Implementation of server-side business logic and REST APIs
Database	MySQL	Persistent storage of patient, doctor, appointment, and medical record data
API Communication	REST API	Communication between the frontend and backend

Authentication	JWT-based authentication	User authentication and role-based access control
Version Control	Git and GitHub	Source code management and team collaboration

3.7 Risks & Mitigations

Risk	Mitigation
Unauthorized access to patient or medical information	Implement authentication and role-based access control.
Incorrect or inconsistent appointment data	Apply validation rules and maintain consistent database operations.
Database failure or data loss	Perform regular database backups and use appropriate recovery procedures.
Integration issues between frontend and backend	Define clear API interfaces and perform integration testing.
Delays in development due to team dependencies	Divide the system into modular components and maintain regular integration of team contributions.
System errors affecting appointment management	Implement error handling, validation, and appropriate test cases for critical workflows.

3.8 Traceability to Requirements

Requirement ID	Requirement / Function	Architectural Component
HMS-F-001 to HMS-F-006	User Authentication	Authentication Service
HMS-F-010 to HMS-F-014	Patient Management	Patient Management Service
HMS-F-020 to HMS-F-024	Doctor Management	Doctor Management Service
HMS-F-030 to HMS-F-036	Appointment Management	Appointment Management Service
HMS-F-040 to HMS-F-044	Medical Record Management	Medical Record Service
HMS-F-040 to HMS-F-044	Administrative Management	Admin Service
HMS-F-060 to HMS-F-063	Data Persistence	Hospital Database
HMS-NF-001 to HMS-NF-007	Non-functional Requirements	System-wide Components
HMS-SR-001 to HMS-SR-008	Security Requirements	Authentication Service and relevant backend components

3.9 Security Architecture

The system will use the following security mechanisms to protect patient, doctor, appointment, and medical record data:

Security Area	Design
Authentication	Users must log in using valid credentials. JWT-based authentication will be used for maintaining authenticated sessions.
Authorization	Role-Based Access Control (RBAC) will restrict access according to the user's role: Patient, Doctor, or Administrator.
Password Security	User passwords will be stored as securely hashed values rather than plain text.
Data Protection	Communication between the frontend and backend will use HTTPS/ TLS to protect data during transmission.
Database Security	Database access will be restricted to the backend services. Sensitive patient information will not be exposed directly to the frontend.
Input Validation	User inputs will be validated to reduce invalid or malicious requests.
Audit & Logging	Important authentication, administrative, and system events will be logged for monitoring and troubleshooting.

4. Design

4.1 Design Overview

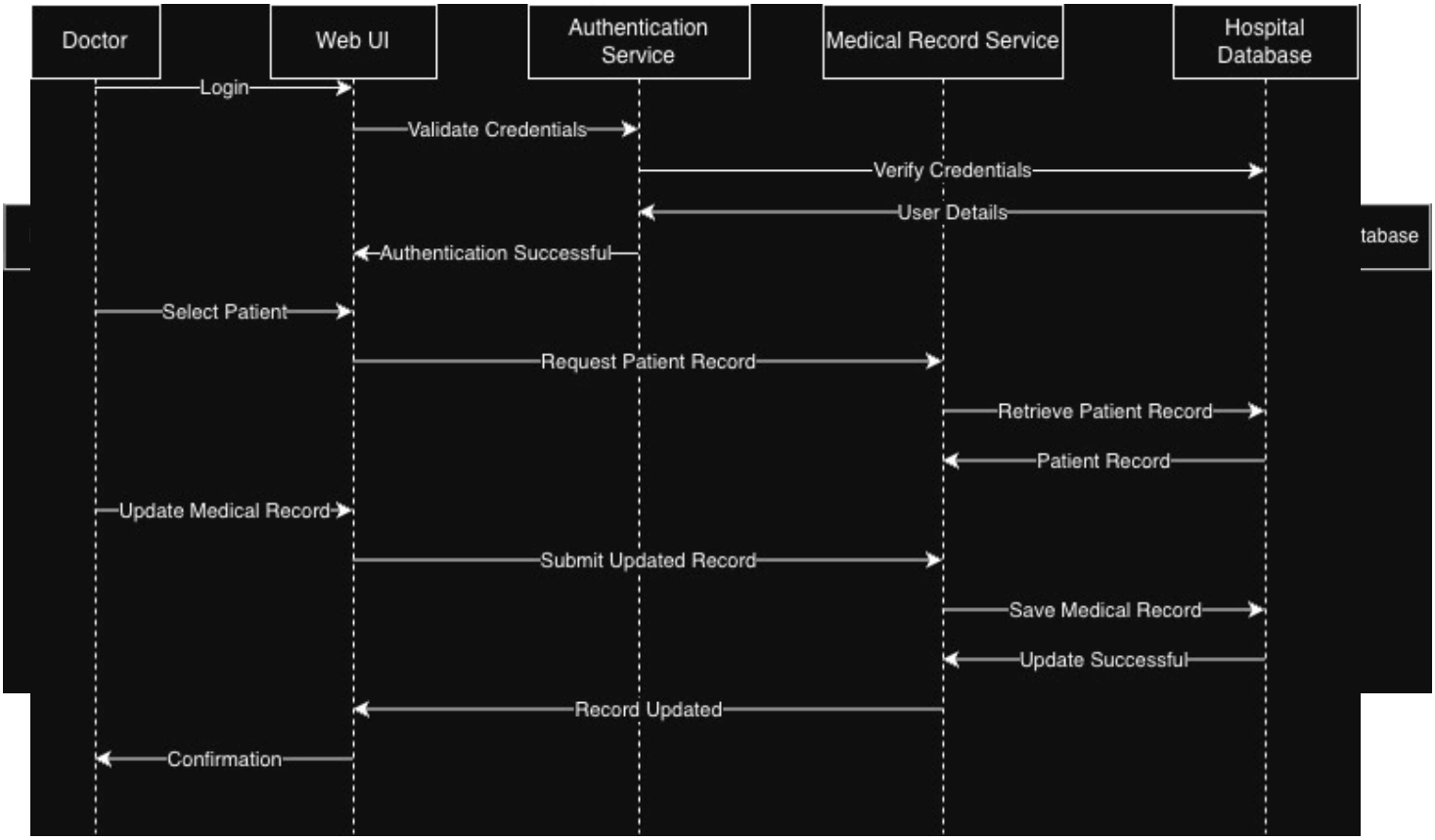
The system design is based on a layered architecture consisting of the presentation, business logic, and data layers.

- **Presentation Layer:** Provides the web interface through which Patients, Doctors, and Administrators interact with the system.
- **Business Logic Layer:** Contains the backend services responsible for authentication, patient management, doctor management, appointments, medical records, and administration.
- **Data Layer:** Uses the Hospital Database to store and retrieve user, patient, doctor, appointment, and medical record information.

The design separates user interaction, business logic, and data management to improve modularity, maintainability, testability, and security.

4.2 UML Sequence Diagrams

At least 2 sequence diagrams covering 2 flows specific to your project



4.3 API Design

The system will use RESTful APIs to enable communication between the frontend and backend services. The APIs will use JSON for request and response data. Protected APIs will require authentication and role-based authorization.

1. Appointment Management API

POST /api/appointments

Purpose: Create a new appointment.

Request:

{

"patientId": 101,

"doctorId": 205,

"date": "2026-10-10",

"time": "10:30"

}

Response:

{

"appointmentId": 501,

"status": "Confirmed"

}

GET /api/appointments/:id

Purpose: Retrieve appointment details.

PUT /api/appointments/:id

Purpose: Update or reschedule an appointment.

DELETE /api/appointments/:id

Purpose: Cancel an appointment.

2. Medical Record API

GET /api/records/:patientId

Purpose: Retrieve the medical records of a patient.

POST /api/records

Purpose: Create a new medical record.

Request:

{

"patientId": 101,

"doctorId": 205,

"diagnosis": "Fever",

"notes": "Follow-up required"

}

Response:

{

"recordId": 301,

"status": "Created"

}

PUT /api/records/:id

Purpose: Update an existing medical record.

Authentication and Authorization:

Protected API endpoints will use JWT-based authentication. Role-Based Access Control (RBAC) will ensure that users can access only the operations permitted for their role.

4.4 Error Handling, Logging & Monitoring

The system will use centralized error handling, logging, and monitoring mechanisms to improve reliability and simplify troubleshooting.

4.4.1 Error Handling

- The system will validate user inputs before processing requests.

- Appropriate HTTP status codes will be returned for API errors.

- Unauthorized requests will return an authentication or authorization error.

- Requests for unavailable or invalid resources will return a suitable error response.

- Database and server errors will be handled without exposing sensitive system information to users.

- User-friendly error messages will be displayed on the frontend.

4.4.2 Logging

- Authentication events such as successful and failed login attempts will be logged.

- Important administrative and system events will be recorded.

- API errors and database errors will be logged for debugging.

- Sensitive information such as passwords will not be stored in logs.

4.4.3 Monitoring

- The system will monitor application errors and failed API requests.

- Database connectivity and backend service availability will be monitored.

- Logs will be reviewed during testing and maintenance to identify recurring errors and system issues.

4.5 UX Design

The system will provide a simple and consistent web-based user interface for Patients, Doctors, and Administrators.

4.5.1 User Interface

- The system will provide separate views and functions based on the user's role.

- Patients will be able to view their information, search for doctors, book appointments, and view appointment details.

- Doctors will be able to view appointments and access and update patient medical records.

- Administrators will be able to manage users, doctors, patients, and system information.

- Navigation will be kept simple and consistent across the application.

- Forms will provide input validation and clear error messages.

- Important actions such as appointment booking, cancellation, and record updates will display confirmation messages.

4.5.2 Usability

- The interface will use clear labels and readable information.

- The system will minimize unnecessary steps for common operations.

- Users will receive appropriate feedback after important actions.

- Unauthorized functions will not be displayed or accessible to users without the required permissions.

4.6 Open Issues & Next Steps

The following aspects may be considered for future development and enhancement of the system:

- Integration with hospital notification systems for appointment reminders.

- Addition of email or SMS notifications for appointment confirmations and updates.

- Expansion of reporting and analytics features for administrators.

- Addition of automated database backup and recovery mechanisms.

- Improvement of system scalability to support a larger number of users and records.

- Further enhancement of security controls based on future system requirements.

- Integration with external hospital systems or healthcare services if required.

These features are outside the current project scope and may be considered in future versions of the system.

5. Appendices

5.1 Glossary

Patient: A registered user who can manage personal information and appointments.

Doctor: A healthcare professional who can view appointments and manage relevant patient medical records.

Administrator: A user responsible for managing system users and administrative functions.

Appointment: A scheduled interaction between a patient and a doctor.

Medical Record: Information related to a patient's consultations, diagnosis, notes, and treatment history.

API: Application Programming Interface used for communication between different software components.

RBAC: Role-Based Access Control used to restrict system access according to user roles.

JWT: JSON Web Token used for authenticated communication between the client and server.

REST: Representational State Transfer, an architectural style used for designing web APIs.

SAD: Software Architecture and Design Specification.

SRS: Software Requirements Specification.

RTM: Requirements Traceability Matrix.

5.2 References

1. Software Requirements Specification (SRS) for the Hospital Patient Record and Appointment Management System.

2. Software Test Plan for the Hospital Patient Record and Appointment Management System.

3. Requirements Traceability Matrix (RTM).

4. UML diagrams and database/ER design documentation prepared for the project.

5.3 Tools and Technologies

- Frontend: React.js

- Backend: Node.js with Express.js

- Database: MySQL

- API Style: REST

- Authentication: JWT

- Version Control: Git and GitHub

- Diagramming Tool: diagrams.net (draw.io)